December 11, 2021

MD-Bench

Maximilian Gaul

MuCoSim Seminar

Preliminary

About

- MD-Bench is a molecular dynamics application which calculates the intersections among particles and how these affect motion
- Simulation system consists of:
  - Number of atoms, each having an initial state and velocity
  - Boundary conditions
- The force of each atom is a sum of its interactions with neighboring atoms

Force modeling

- In MD-Bench, the Lennard-Jones potential is used for calculating the forces between pairs of particles [1]
  - In our case, particles are electronically neutral atoms
- This function models repulsive as well as attractive interactions

$$V_{LJ}(r) = 4\epsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]$$

The Lennard-Jones potential I

$$V_{LJ}(r) = 4\epsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]$$

- Where
  - **r** is the distance between two interacting atoms
  - ϵ is the dispersion energy
  - σ is the distance at which the potential **V** is equal to zero

The Lennard-Jones potential II

- Behavior of the Lennard-Jones potential with increasing distance **r** in **Å** (Ångström)

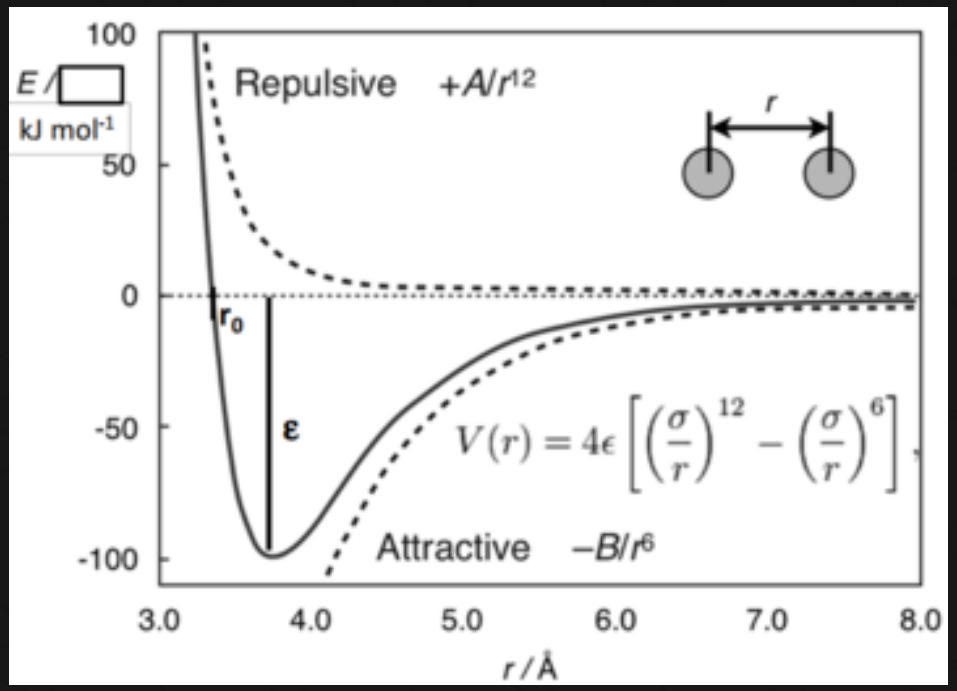


Figure: Lennard Jones Potential

The Lennard-Jones potential III

- The Lennard-Jones potential is a simplified model but still describes the essential aspects of particle dynamics
- We observe that particles
  - repel each other at close distance
  - attract each other at medium distances and have
  - close to zero interaction at large distances

Testsystem

Cluster

- Host/Clustername: tinyGPU.tg086
- Cluster Info
  - https://hpc.fau.de/systems-services/systems-documentation-instructions/clusters/tinygpu-cluster/
- CPU type: 2x Intel Xeon Gold 6226R @2.9 GHz = 32 cores with optional SMT
- GPU type: 8x NVIDIA Geforce RTX3080
- Memory capacity: 384 GB RAM
- Software Environment:
  - Compiler: NVIDIA NVCC Build cuda_11.2.r11.2
  - Operating System: Ubuntu 20.04.3 LTS

MD-Bench

Force calculation I

- Calculation of the Lennard-Jones potential is done for pairs of atoms (pseudocode):

```
for atom in atoms:
    neighbors = neighbor.neighbors[atom]
    force = 0.0
    for neighbor in neighbors:
        radius = calc_radius(...)
        if radius < close_enough:
            force += calc_force(...)
    forces[atom] += force
```

Force calculation II

- It is time consuming and unnecessary to calculate the force for *every* pair of atoms if they are so far apart that the force is close to 0 anyways

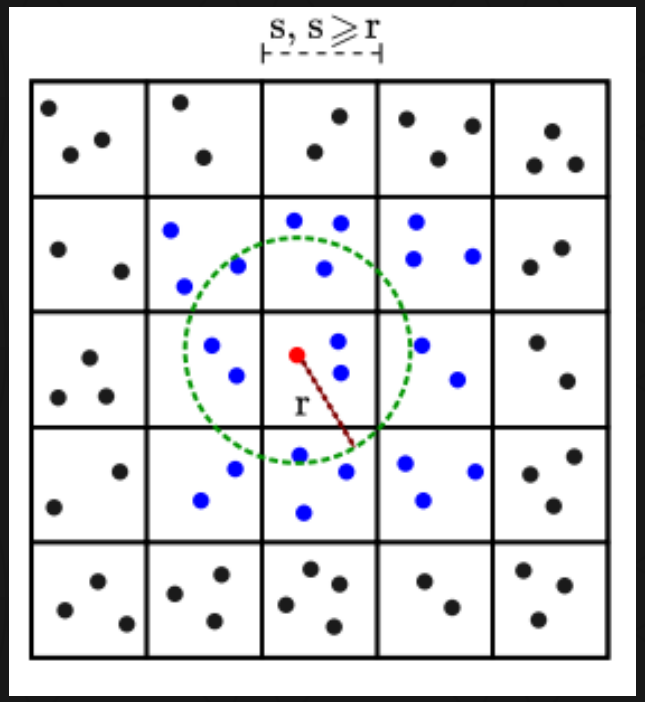


Figure: Neighbor list [1]

Configuration

- Clone the repository

```
foo@bar:~$ git clone ↙
    ↳ https://github.com/RRZE-HPC/MD-Bench.git
foo@bar:~$ cd MD-Bench
foo@bar:~/MD-Bench$ git checkout mucosim_cuda
```

- Load modules

```
foo@bar:~/MD-Bench$ module load cuda likwid
```

- Change
  - TAG variable in config.mk to TAG ?= NVCC
  - Compiler flags in include_NVCC.mk

Run Software

- Compile and execute

```
foo@bar:~/MD-Bench$ make
foo@bar:~/MD-Bench$ ./MDBench-NVCC
```

- MD-Bench will output temperature and pressure data as well as performance in million atom updates per second
- Modify number of runs with argument
  -n <nmbr>

Program Output

- With unmodified parameters

step	temp	pressure
0	1.440000e+00	1.215639e+00
100	8.200895e−01	6.923143e−01
200	7.961495e−01	6.721043e−01

```
Force field: lj
Data layout for positions: AoS
Using double precision floating point.
```

```
System: 131072 atoms 47265 ghost atoms, ↙
    ↳ Steps: 200
TOTAL 6.83s FORCE 0.73s NEIGH 3.07s REST 3.04s
```

```
Performance: 3.84 million atom updates per ↙
    ↳ second
```

Scaling runs

CUDA I

- CUDA programming model
  - Basic execution size is a *thread* which is executed by one CUDA core
  - Threads are grouped together as *blocks*, these are executed by streaming multiprocessors (*SM*)
  - Resources on an SM are allocated on a *warp*-size level (32 threads)
  - Warps are dispatched to execution units of GPU

CUDA II

- src/force.c was refactored to support CUDA kernel functions and distribution on multiple Threads of a GPU
- Force-calculation is parallelized on an atom-level
- Each atom computes the force for every other atom in its neighbor list (around 100)

Scaling threads I

- Default number of atoms: *131072*
- Around 10^7 force computations per run
- Number of blocks is calculated by (`num_threads` is fixed):

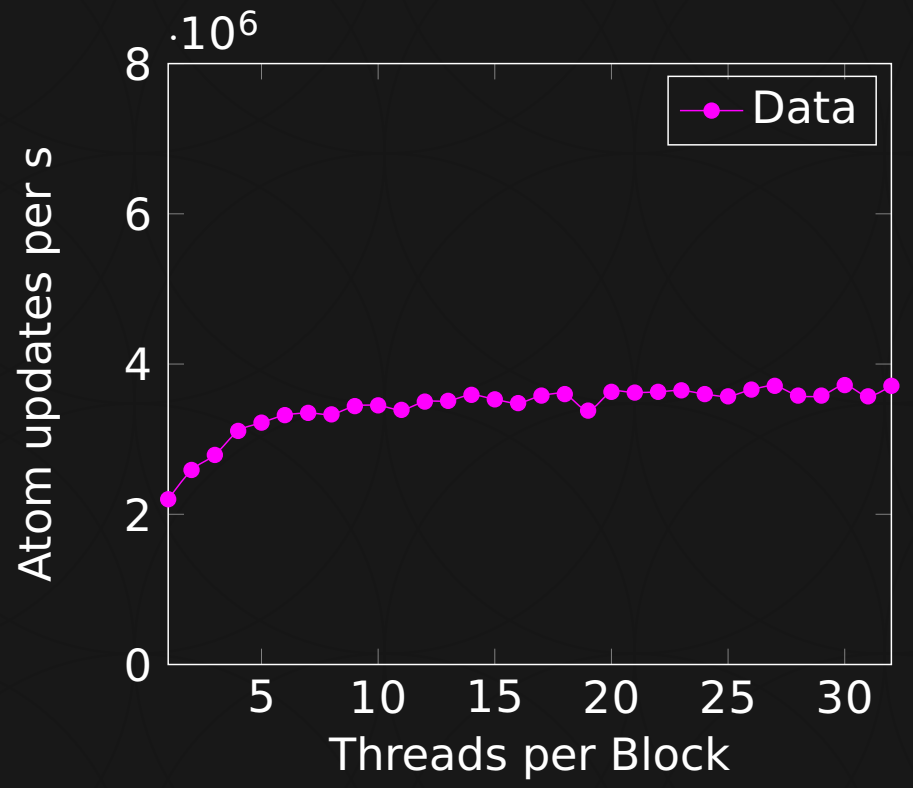
```
blocks = ceil( 131072 / num_threads )
```

- Run on cluster with one GPU:

```
foo@bar:~/MD-Bench$ srun.tinygpu -N 1 -n 1 ↙
    ↳ --gres=gpu:1 --pty /bin/bash -l
foo@bar:~/MD-Bench$ NUM_THREADS=X ↙
    ↳ ./MD-Bench-NVCC -n 50
```

Scaling threads II

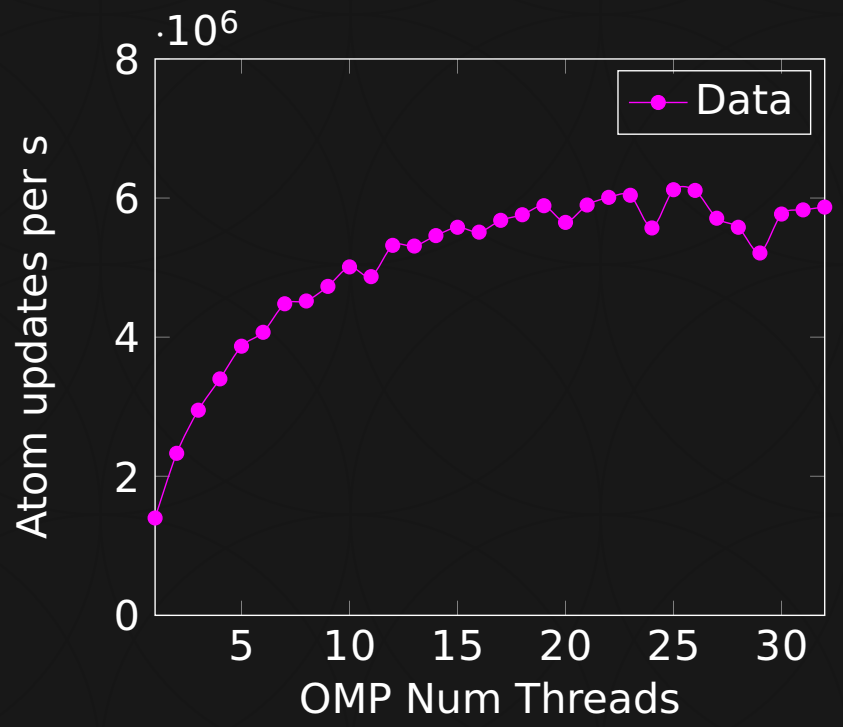
- As resources are allocated on a warp-size level, we expect to have highest performance for 32 threads per block



Scaling threads III

- Performance improvement stagnates
- But how does the CPU perform?

```
foo@bar:~/MD-Bench$ srun.tinyfat --ntasks=1 ↙
    ↳ --cpus-per-task=32 --pty /bin/bash -l
```



Measurements

Memory Bandwidth I

- Using the NVIDIA *nvprof* profiling tool via

```
foo@bar:~/MD-Bench$ nvprof --print-gpu-trace ↙
    ↳ ./MDBench-NVCC -n 50
```

enables detailed metrics on memory throughput

Duration	Size	Throughput	SrcMemType	DstMemType	Name
899.68us	4.1199MB	4.4719GB/s	Pageable	Device	[CUDA memcpy HtoD]
87.904us	1.0000MB	11.109GB/s	Pageable	Device	[CUDA memcpy HtoD]
87.616us	1.0000MB	11.146GB/s	Pageable	Device	[CUDA memcpy HtoD]
87.808us	1.0000MB	11.122GB/s	Pageable	Device	[CUDA memcpy HtoD]
60.640us	703.13KB	11.058GB/s	Pageable	Device	[CUDA memcpy HtoD]
896ns	128B	136.24MB/s	Pageable	Device	[CUDA memcpy HtoD]
864ns	128B	141.29MB/s	Pageable	Device	[CUDA memcpy HtoD]
864ns	128B	141.29MB/s	Pageable	Device	[CUDA memcpy HtoD]
11.534ms	50.000MB	4.2335GB/s	Pageable	Device	[CUDA memcpy HtoD]
45.248us	512.00KB	10.791GB/s	Pageable	Device	[CUDA memcpy HtoD]
19.917ms	-	-	-	-	calcforce() [132]
80.896us	1.0000MB	12.072GB/s	Device	Pageable	[CUDA memcpy DtoH]
80.736us	1.0000MB	12.096GB/s	Device	Pageable	[CUDA memcpy DtoH]
80.480us	1.0000MB	12.134GB/s	Device	Pageable	[CUDA memcpy DtoH]

Memory Bandwith II

- Together with analysis of the output profile in NVIDIA Nsight Systems:

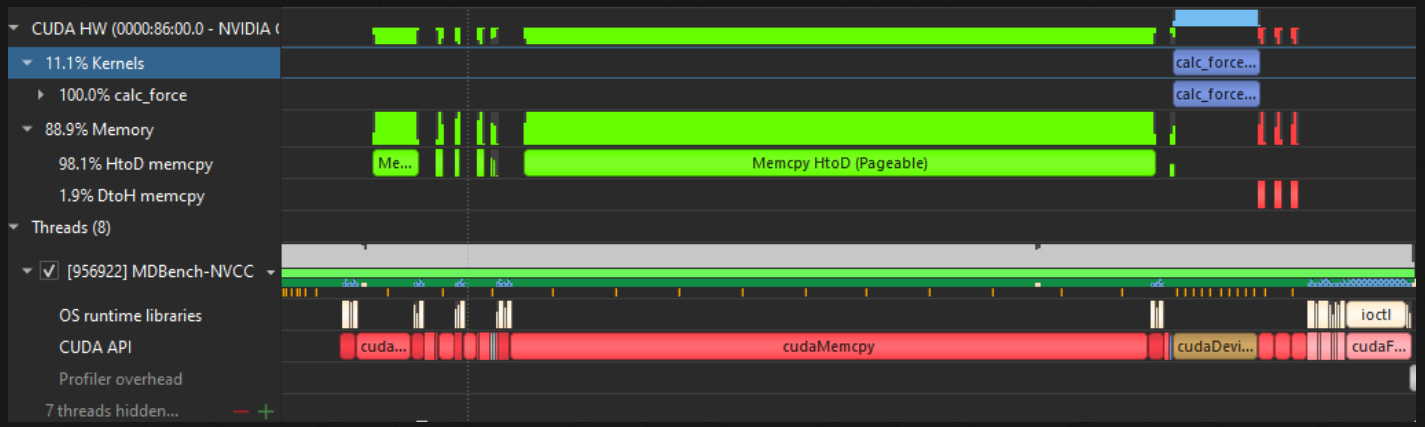


Figure: NVIDIA Nsight Systems

Memory Bandwith III

- and the roofline model from

```
foo@bar:~/MD-Bench$ ncu --set full -o ↙
    ↳ profile ./MDBench-NVCC
```

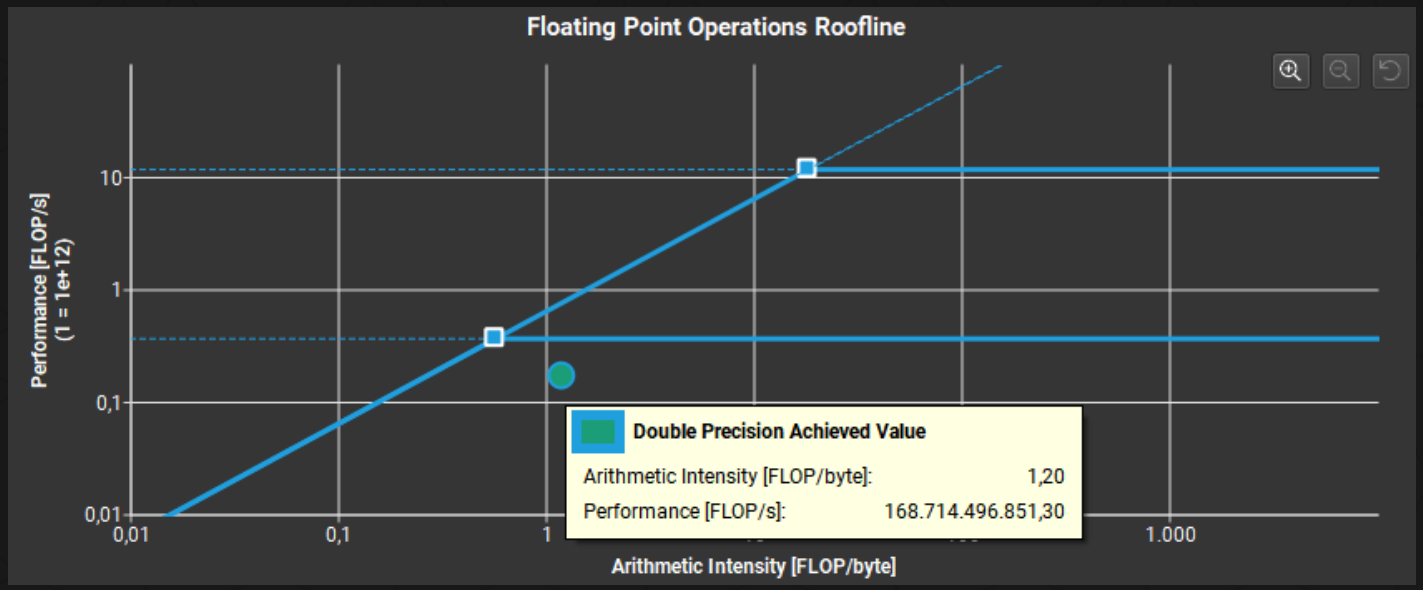
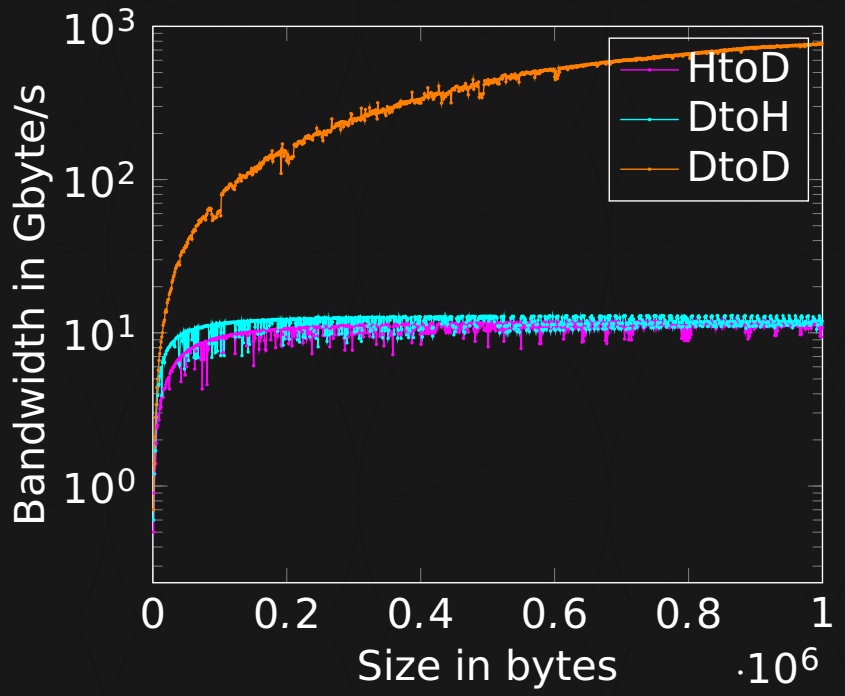


Figure: NVIDIA Nsight Compute Roofline Model

Memory Bandwidth IV

- we observe that memory transfers from CPU to GPU and vice versa are slow
- Using NVIDIA's CUDA samples we are able to evaluate maximum transfer speeds in practice



Future Topics

- Low performance is not caused by MD-Bench memory layout but by hardware limitations
- To increase performance we have to avoid DtoH and HtoD memory accesses as much as possible
  - Change memory layout from Structure of Arrays to Array of Structures
  - Avoid unnecessary copies
  - Port other parts of MD-Bench to CUDA such that pretty much everything can run inside the GPU

Bibliography I

Machado, R. R. L., Schmitt, J., Eibl, S., Eitzinger, J., LeiSSa, R., Hack, S., Pérard-Gayot, A., Membarth, R., and Köstler, H.
tinymd: Mapping molecular dynamics simulations to heterogeneous hardware using partial evaluation.
Journal of Computational Science 54 (2021), 101425.